

```

const CACHE_NAME = 'mirror-v1';

// Files to cache for offline use
const STATIC_ASSETS = [
  '/index.html',
  '/manifest.json'
];

// Install - cache all static assets
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      return cache.addAll(STATIC_ASSETS);
    })
  );
  self.skipWaiting();
});

// Activate - clean up old caches
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(keys =>
      Promise.all(
        keys.filter(key => key !== CACHE_NAME).map(key => caches.delete(key))
      )
    )
  );
  self.clients.claim();
});

// Fetch - serve from cache when offline, network when online
self.addEventListener('fetch', event => {
  // For API calls (Anthropic), always go to network - never cache
  if (event.request.url.includes('api.anthropic.com')) {
    return;
  }

  // For Google Fonts, try network first, fall back to cache
  if (event.request.url.includes('fonts.googleapis.com') ||
    event.request.url.includes('fonts.gstatic.com')) {
    event.respondWith(
      caches.open(CACHE_NAME).then(cache =>
        fetch(event.request)
          .then(response => {

```

```

        cache.put(event.request, response.clone());
        return response;
    })
    .catch(() => caches.match(event.request))
  )
);
return;
}

// For everything else - cache first, then network
event.respondWith(
  caches.match(event.request).then(cached => {
    if (cached) return cached;
    return fetch(event.request).then(response => {
      if (response && response.status === 200) {
        const copy = response.clone();
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, copy));
      }
      return response;
    });
  })
);
});

```